</ Data Mining:
Practical Python
for Beginners

/>

} /> [

Instructor:
Fateme Khodabande

fatemever2001@gmail.com

1 0 1 1 0 1 1 0 1 1 0 1 1 0 0 1 1 0 1 1 0 1 1 0 1 1 0 1 1 1 0 1 1 0 1 1 0 1 1 1 1 1 0 1

NumPy

1 0 1 1 0 1 1 0 1 1 0 1 1 0 0 1 1 0 1 1 0 1 1 0 1 1 0 1 1 1 0 1 1 0 1 1 0 1 1 1 1 1 0 1

Introduction

- Vectors, matrices, and arrays of higher dimensions are essential tools in numerical computing. When a computation must be repeated for a set of input values, it is natural and advantageous to represent the data as arrays and the computation in terms of array operations.
- Vectorized computing eliminates the need for many explicit loops over the array elements by applying batch operations on the array data.
- Vectorized computations can therefore be significantly faster than sequential element-by-element computations. This is particularly important in an interpreted language such as Python, where looping over arrays element by element entails a significant performance overhead.

NumPy

- In Python's scientific computing environment, efficient data structures for working with arrays are provided by the NumPy library. The core of NumPy is implemented in C and provides efficient functions for manipulating and processing arrays.
- NumPy arrays are homogenous and typed arrays of fixed size. Homogenous means that all elements in the array have the same data type. Fixed size means that an array cannot be resized (without creating a new array). For these and other reasons, operations and functions acting on NumPy arrays can be much more efficient than those using Python lists.
- NumPy also provides a large collection of basic operators and functions that act on these data structures, as well as submodules with higher-level algorithms such as linear algebra and fast Fourier transform.

1 0 1 1 0 1 1 0 1 1 0 1 1 0 0 1 1 0 1 1 0 1 1 0 1 1 0 1 1 1 0 1 1 0 1 1 0 1 1 1 1 1 0 1

The NumPy Array Object

- The core of the NumPy library is the data structures for representing multidimensional arrays of homogeneous data. (Homogeneous refers to all elements in an array having the same data type)
- The main data structure for multidimensional arrays in NumPy is the ndarray class.
- In addition to the data stored in the array, this data structure also contains important metadata about the array, such as its shape, size, data type, and other attributes.

Attribute	Description
`Shape`	A tuple that contains the number of elements (i.e., the length) for each dimension (axis) of the array.
`Size`	The total number elements in the array.
`Ndim`	Number of dimensions (axes).
`nbytes`	Number of bytes used to store the data.
`dtype`	The data type of the elements in the array.

1 0 1 1 0 1 1 0 1 1 0 1 1 0 0 1 1 0 1 1 0 1 1 0 1 1 0 1 1 0 1 1 0 1 1 1 1 0 1

Numerical Data Types Available in NumPy

- Nonnumerical data types, such as strings, objects, and user-defined compound types, are also supported.
- Creating an array with the given type
- Changing the type of the array's elements

dtype	Variants	Description
int	int8, int16, int32, int64	Integers
uint	uint8, uint16, uint32, uint64	Unsigned (nonnegative) integers
bool	Bool	Boolean (True or False)
float	float16, float32, float64, float128	Floating-point numbers
complex	complex64, complex128, complex256	Complex-valued floating-point numbers

1 0 1 1 0 1 1 0 1 1 0 1 1 0 0 1 1 0 1 1 0 1 1 0 1 1 0 1 1 0 1 1 0 1 1 1 1 1 0 1

NumPy Functions: Generating Arrays

Table 2-3. *Summary of NumPy Functions for Generating Arrays*

Function Name	Type of Array
`np.array`	Creates an array for which the elements are given by an array-like object, which, for example, can be a (nested) Python list, a tuple, an iterable sequence, or another `ndarray` instance.
`np.zeros`	Creates an array with the specified dimensions and data type that is filled with zeros.
`np.ones`	Creates an array with the specified dimensions and data type that is filled with ones.
`np.diag`	Creates a diagonal array with specified values along the diagonal and zeros elsewhere.
`np.arange`	Creates an array with evenly spaced values between the specified start, end, and increment values.
`np.linspace`	Creates an array with evenly spaced values between specified start and end values, using a specified number of elements.
`np.logspace`	Creates an array with values that are logarithmically spaced between the given start and end values.

Table 2-3. *Summary of NumPy Functions for Generating Arrays*

Function Name	Type of Array
`np.meshgrid`	Generates coordinate matrices (and higher-dimensional coordinate arrays) from one-dimensional coordinate vectors.
`np.fromfunction`	Creates an array and fills it with values specified by a given function, which is evaluated for each combination of indices for the given array size.
`np.fromfile`	Creates an array with the data from a binary (or text) file. NumPy also provides a corresponding function `np.tofile` with which NumPy arrays can be stored to disk and later read back using `np.fromfile`.
`np.genfromtxt`,`np.loadtxt`	Create an array from data read from a text file, for example, a comma-separated value (CSV) file. The function `np.genfromtxt` also supports data files with missing values.
`np.random.rand`	Generates an array with random numbers that are uniformly distributed between 0 and 1. Other types of distributions are also available in the `np.random` module.

1 0 1 1 0 1 1 0 1 1 0 1 1 0 0 1 1 0 1 1 0 1 1 0 1 1 0 1 1 0 1 1 0 1 1 1 1 1 0 1

NumPy Functions: Indexing & Slicing

Expression	Description
`a[m]`	Select element at index *m*, where *m* is an integer (start counting form 0).
`a[-m]`	Select the *n*th element from the end of the list, where *n* is an integer. The last element in the list is addressed as –1, the second to last element as –2, and so on.
`a[m:n]`	Select elements with index starting at *m* and ending at *n* – 1 (*m* and *n* are integers).
`a[:]` or `a[0:-1]`	Select all elements in the given axis.
`a[:n]`	Select elements starting with index 0 and going up to index *n* – 1 (integer).
`a[m:]` or `a[m:-1]`	Select elements starting with index *m* (integer) and going up to the last element in the array.
`a[m:n:p]`	Select elements with index *m* through *n* (exclusive), with increment *p*.
`a[::-1]`	Select all the elements, in reverse order.

1 0 1 1 0 1 1 0 1 1 0 1 1 0 0 1 1 0 1 1 0 1 1 0 1 1 0 1 1 0 1 1 0 1 1 1 1 1 0 1

NumPy Functions: Reshaping and Resizing

Function/Method	Description
`np.reshape`, `np.ndarray.reshape`	Reshape an N-dimensional array. The total number of elements must remain the same.
`np.ndarray.flatten`	Creates a copy of an N-dimensional array, and reinterpret it as a one-dimensional array (i.e., all dimensions are collapsed into one).
`np.ravel`, `np.ndarray.ravel`	Create a view (if possible, otherwise a copy) of an N-dimensional array in which it is interpreted as a one-dimensional array.
`np.squeeze`	Removes axes with length 1.
`np.expand_dims`, `np.newaxis`	Add a new axis (dimension) of length 1 to an array, where `np.newaxis` is used with array indexing.
`np.transpose`, `np.ndarray.transpose`, `np.ndarray.T`	Transpose the array. The transpose operation corresponds to reversing (or more generally, permuting) the axes of the array.
`np.hstack`	Stacks a list of arrays horizontally (along axis 1): for example, given a list of column vectors, appends the columns to form a matrix.
`np.vstack`	Stacks a list of arrays vertically (along axis 0): for example, given a list of row vectors, appends the rows to form a matrix.

Function/Method	Description
`np.dstack`	Stacks arrays depth-wise (along axis 2).
`np.concatenate`	Creates a new array by appending arrays after each other, along a given axis.
`np.resize`	Resizes an array. Creates a new copy of the original array, with the requested size. If necessary, the original array will be repeated to fill up the new array.
`np.append`	Appends an element to an array. Creates a new copy of the array.
`np.insert`	Inserts a new element at a given position. Creates a new copy of the array.
`np.delete`	Deletes an element at a given position. Creates a new copy of the array.

1 0 1 1 0 1 1 0 1 1 0 1 1 0 0 1 1 0 1 1 0 1 1 0 1 1 0 1 1 1 0 1 1 0 1 1 0 1 1 1 1 1 0 1

NumPy Functions: Elementwise Functions

NumPy Function	Description
np.cos, np.sin, np.tan	Trigonometric functions.
np.arccos, np.arcsin, np.arctan	Inverse trigonometric functions.
np.cosh, np.sinh, np.tanh	Hyperbolic trigonometric functions.
np.arccosh, np.arcsinh, np.arctanh	Inverse hyperbolic trigonometric functions.
np.sqrt	Square root.
np.exp	Exponential.
np.log, np.log2, np.log10	Logarithms of base e, 2, and 10, respectively.

NumPy Function	Description
np.add, np.subtract, np.multiply, np.divide	Addition, subtraction, multiplication, and division of two NumPy arrays.
np.power	Raises first input argument to the power of the second input argument (applied elementwise).
np.remainder	The remainder of division.
np.reciprocal	The reciprocal (inverse) of each element.
np.real, np.imag, np.conj	The real part, imaginary part, and the complex conjugate of the elements in the input arrays.
np.sign, np.abs	The sign and the absolute value.
np.floor, np.ceil, np.rint	Convert to integer values.
np.round	Rounds to a given number of decimals.

1 0 1 1 0 1 1 0 1 1 0 1 1 0 0 1 1 0 1 1 0 1 1 0 1 1 0 1 1 1 0 1 1 0 1 1 0 1 1 1 1 1 0 1

NumPy Functions: Aggregate Functions

NumPy Function	Description
`np.mean`	The average of all values in the array.
`np.std`	Standard deviation.
`np.var`	Variance.
`np.sum`	Sum of all elements.
`np.prod`	Product of all elements.
`np.cumsum`	Cumulative sum of all elements.
`np.cumprod`	Cumulative product of all elements.
`np.min, np.max`	The minimum/maximum value in an array.
`np.argmin, np.argmax`	The index of the minimum/maximum value in an array.
`np.all`	Returns True if all elements in the argument array are nonzero.
`np.any`	Returns True if any of the elements in the argument array is nonzero.

1 0 1 1 0 1 1 0 1 1 0 1 1 0 0 1 1 0 1 1 0 1 1 0 1 1 0 1 1 1 0 1 1 0 1 1 0 1 1 1 1 1 0 1

NumPy Functions: Boolean Arrays and Conditional Expressions

Function	Description
`np.where`	Chooses values from two arrays depending on the value of a condition array.
`np.choose`	Chooses values from a list of arrays depending on the values of a given index array.
`np.select`	Chooses values from a list of arrays depending on a list of conditions.
`np.nonzero`	Returns an array with indices of nonzero elements.
`np.logical_and`	Performs an elementwise AND operation.
`np.logical_or, np.logical_xor`	Elementwise OR/XOR operations.
`np.logical_not`	Elementwise NOT operation (inverting).

1 0 1 1 0 1 1 0 1 1 0 1 1 0 0 1 1 0 1 1 0 1 1 0 1 1 0 1 1 1 0 1 1 0 1 1 0 1 1 1 1 1 0 1

NumPy Functions: Set Operations

Function	Description
`np.unique`	Creates a new array with unique elements, where each value only appears once.
`np.in1d`	Tests for the existence of an array of elements in another array.
`np.intersect1d`	Returns an array with elements that are contained in two given arrays.
`np.setdiff1d`	Returns an array with elements that are contained in one, but not the other, of two given arrays.
`np.union1d`	Returns an array with elements that are contained in either, or both, of two given arrays.

1 0 1 1 0 1 1 0 1 1 0 1 1 0 0 1 1 0 1 1 0 1 1 0 1 1 0 1 1 1 0 1 1 0 1 1 0 1 1 1 1 1 0 1

NumPy Functions: Operations on Arrays

Function	Description
`np.transpose`, `np.ndarray.transpose`, `np.ndarray.T`	The transpose (reverse axes) of an array.
`np.fliplr/np.flipud`	Reverse the elements in each row/column.
`np.rot90`	Rotates the elements along the first two axes by 90 degrees.
`np.sort`, `np.ndarray.sort`	Sort the elements of an array along a given specified axis (which default to the last axis of the array). The `np.ndarray` method sort performs the sorting in place, modifying the input array.

1 0 1 1 0 1 1 0 1 1 0 1 1 0 0 1 1 0 1 1 0 1 1 0 1 1 0 1 1 1 0 1 1 0 1 1 0 1 1 1 1 1 0 1

NumPy Functions: Matrix and Vector Operations

NumPy Function	Description
`np.dot`	Matrix multiplication (dot product) between two given arrays representing vectors, arrays, or tensors.
`np.inner`	Scalar multiplication (inner product) between two arrays representing vectors.
`np.cross`	The cross product between two arrays that represent vectors.
`np.tensordot`	Dot product along specified axes of multidimensional arrays.
`np.outer`	Outer product (tensor product of vectors) between two arrays representing vectors.
`np.kron`	Kronecker product (tensor product of matrices) between arrays representing matrices and higher-dimensional arrays.
`np.einsum`	Evaluates Einstein's summation convention for multidimensional arrays.

1 0 1 1 0 1 1 0 1 1 0 1 1 0 0 1 1 0 1 1 0 1 1 0 1 1 0 1 1 1 0 1 1 0 1 1 0 1 1 1 1 1 0 1

Further Reading

NumPy Official Documentation

1 0 1 1 0 1 1 0 1 1 0 1 1 0 0 1 1 0 1 1 0 1 1 0 1 1 0 1 1 1 0 1 1 0 1 1 0 1 1 1 1 1 0 1

Pandas

1 0 1 1 0 1 1 0 1 1 0 1 1 0 0 1 1 0 1 1 0 1 1 0 1 1 0 1 1 1 0 1 1 0 1 1 0 1 1 1 1 1 0 1

Introduction

- This library provides convenient data structures for representing series and tables of data and makes it easy to transform, split, merge, and convert data.
- The Pandas library builds on top of NumPy and complements it with features that are particularly useful when handling data, such as labeled indexing, hierarchical indices, alignment of data for comparison and merging of datasets, handling of missing data, and much more.

1 0 1 1 0 1 1 0 1 1 0 1 1 0 0 1 1 0 1 1 0 1 1 0 1 1 0 1 1 1 0 1 1 0 1 1 0 1 1 1 1 1 0 1

Data Frames and Series

The main focus of this chapter is the Pandas library for data analysis, and we begin here with an introduction to this library. The Pandas library mainly provides data structures and methods for representing and manipulating data. The two main data structures in Pandas are the Series and Data Frame objects, which are used to represent data series and tabular data, respectively. Both of these objects have an index for accessing elements or rows in the data represented by the object. By default, the indices are integers starting from zero, like NumPy arrays, but it is also possible to use any sequence of identifiers as index.

Series Operation

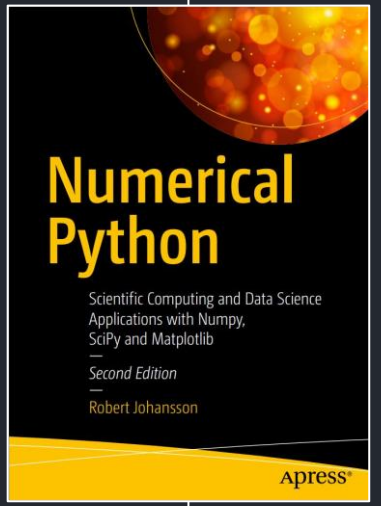
- Converting to Numpy Array
- Adding Nominal Index and Name to The Serie
- Accessing Values With Index
- Descriptive Statistics
- Combining With Matplotlib

1 0 1 1 0 1 1 0 1 1 0 1 1 0 0 1 1 0 1 1 0 1 1 0 1 1 0 1 1 1 0 1 1 0 1 1 0 1 1 1 1 1 0 1

Data Frame Operation

- Creating a Data Frame
- Adding New Rows to The Data Frame
- Locating
- General Information
- Exporting as a CSV or XLSX

</ Reference: Numerical Python />



https://doi.org/10.1007/978-1-4842-4246-9

1 0 1 1 0 1 1 0 1 1 0 1 1 0 0 1 1 0 1 1 0 1 1 0 1 1 0 1 1 1 0 1 1 0 1 1 0 1 1 1 1 1 0 1